3.1.6 High Precision Calculations

When you do calculations with high precision numbers, *Mathematica* always keeps track of the precision of your results. *Mathematica* tries to give you results that are as accurate as possible, given the precision of the input you provided.

Here is the numerical value of π^2 to 30 significant figures.

```
In[1]:= N[ Pi^2, 30 ]
Out[1]= 9.86960440108935861883449099988
```

Operations like these give you results that have the same precision as your input.

```
In[2]:= 2 Sqrt[ % ] + 1
Out[2]= 7.2831853071795864769252867665590
```

If you multiply by a number with 20-digit precision, *Mathematica* gives you a 20-digit result.

```
In[3]:= % N[Sqrt[2], 20]
Out[3]= 10.299979438689827542833
```

When you multiply several numbers together, the general rule is that the result you get has the same precision as the least precise one of the numbers. When you add numbers, it is the *accuracy* of each number that enters, rather than its precision.

You will often find that you get results that are less precise than the input you gave. One common reason for this is "roundoff error": if you subtract two nearby numbers, the result is necessarily less precise than the numbers themselves. *Mathematica* always keeps track of the precision of the results you get.

Even though each number is given to high precision, their difference has much lower precision.

```
In[4]:= 1.1111111111111111111 -
        1.1111111111111111000
                  -16
Out[4]= 1.11 10
```

Here is the numerical value of log(3) to 30 significant figures.

```
In[5]:= N[ Log[3], 30 ]
Out[5]= 1.09861228866810969139524523692
```

The exponential of the finite precision approximation to log(3) differs from 3 in the last decimal place.

```
In[6]:= Exp[ % ]
Out[6]= 3.
```

The difference from the exact result is given to only one significant figure.

```
In[7]:= % - 3
                -33
Out[7]= -1. 10
```

The precision of the output from a function can depend in a complicated way on the precision of the input. Functions that vary rapidly typically give less precise output, since the variation of the output associated with uncertainties in the input is larger. Functions that are close to constants can actually give output that is more

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; http://www.wolfram.com/; 1-800-441-6284.

precise than their input.

Functions like Sin that vary rapidly typically give output that is less precise than their input.

```
In[8]:= Sin[111111111.0000000000000000]
Out[8]= -0.29753510333494323
```

Here is e^{-40} to 20 digit precision.

```
In[9]:= N[Exp[-40], 20]
                                      -18
Out[9]= 4.2483542552915889953 10
```

The result you get by adding 1 has a higher precision.

```
In[10]:= 1 + %
Out[10]= 1.0000000000000000042483542552915889953293
```

It is worth realizing that different ways of doing the same calculation can end up giving you results with very different precisions. Typically, if you once lose precision in a calculation, it is essentially impossible to regain it: in losing precision, you are effectively losing information about your result.

Here is a 40-digit number that is close to 1.

```
In[11]:= x = N[1 - 10^-30, 40]
Out[11]= 0.999999999999999999999999999999
```

Adding 1 to it gives another 40-digit number.

```
In[12]:= 1 + x
Out[12]= 1.999999999999999999999999999999
```

Most of the original precision has been maintained.

```
In[13]:= Precision[%]
Out[13]= 44
```

This way of computing 1 + x loses precision.

```
In[14]:= (x^2 - 1) / (x - 1)
Out[14]= 2.
```

The result obtained in this way has quite low precision.

```
In[15]:= Precision[%]
Out[15]= 15
```

The internal algorithms that *Mathematica* uses to evaluate mathematical functions are set up to maintain as much precision as possible. In most cases, built-in *Mathematica* functions will give you results that have as much precision as can be justified on the basis of your input. In some cases, however, it is simply impractical to do this, and *Mathematica* will give you results that have lower precision. If you give higher precision input, *Mathematica* will use higher precision in its internal calculations, and you will usually be able to get a higher precision result.

The fact that different ways of doing the same calculation can give you different numerical answers, means, among other things, that comparisons between approximate real numbers must be treated with care. In testing whether two real numbers are "equal", *Mathematica* effectively finds their difference, and tests whether the result is "consistent with zero" to the precision given.

These numbers are equal to the precision given.

```
In[16]:= 3 == 3.000000000000000000
Out[16]= True
```